🌐 O que é HTTP?

(HyperText Transfer Protocol)

O **HTTP (HyperText Transfer Protocol)** é um protocolo de camada de aplicação que permite a transferência de dados na web. Ele funciona em um modelo **Cliente-Servidor**:

1. O **Cliente** (seu navegador ou sua aplicação) **envia** uma **Requisição (Request)**.

2. O **Servidor** (onde o site/API está hospedado) processa e **envia** uma **Resposta (Response)**.

Ponto chave: O HTTP é *stateless* (sem estado), o que significa que o servidor não guarda memória de requisições anteriores sozinho. **Por isso usamos cookies ou tokens** (como JWT) para manter um usuário logado.

🛠️ Como é usado?

Ele é usado sempre que você acessa uma **URL**. A comunicação **é baseada em mensagens de texto estruturadas**. Uma requisição típica contém:

- **Método:** A ação a ser feita;
- **Path:** O caminho do recurso (ex: /usuarios/1);
- **Headers:** Metadados (tipo de arquivo, tokens de autenticação);
- **Body:** O conteúdo (usado em POST/PUT, geralmente em JSON).

🛡️ HTTP vs. HTTPS (O "S" de Segurança)

- **HTTP:** Os dados são enviados de forma clara. Se alguém interceptar a conexão (ataque *Man-in-the-Middle*), consegue ler senhas e dados sensíveis.
- **HTTPS (HTTP Secure):** Utiliza um protocolo de criptografia chamado **TLS (Transport Layer Security)**.

Mesmo que alguém intercepte os dados no HTTPS, verá apenas um amontoado de caracteres sem sentido. Para sites modernos e APIs, o HTTPS é obrigatório.

O que são os Headers (Cabeçalhos)?

Eles são como a "etiqueta" de uma encomenda. Eles não são o produto (o corpo da mensagem), mas dizem quem enviou, para onde vai, o que tem dentro e como tratar aquilo.

1. Request Headers (O que o Cliente envia)
Authorization: Aqui vai o token (como o Bearer Token) para provar que você está logada.
Content-Type: Avisa ao servidor o formato do que você está enviando (ex: application/json).
User-Agent: Identifica qual navegador ou sistema está fazendo a requisição.

2. Response Headers (O que o Servidor responde)
Set-Cookie: O servidor pede para o seu navegador guardar uma informação
Access-Control-Allow-Origin: Crucial para resolver erros de **CORS** (quando seu front-end tenta falar com um back-end em domínios diferentes).
Cache-Control: Diz por quanto tempo o navegador pode guardar aquela imagem ou arquivo sem precisar pedir de novo.

Os 9 Métodos HTTP (Verbos)

Para organizar as intenções da requisição, usamos métodos específicos

Get	Busca informações (não altera nada no servidor).
Post	Cria um novo recurso.
Put	Atualiza um recurso inteiro (substitui o que existe).
Patch	Atualiza apenas uma parte do recurso.
Delete	Remove um recurso.
Head	Igual ao GET, mas pede apenas os cabeçalhos (sem o corpo).
Options	Verifica quais métodos são permitidos para aquela URL (comum em erros de CORS).
Connect	Estabelece um túnel para o servidor (usado em proxies/SSL).
Trace	Usado para fins de teste, ecoando a requisição de volta.

Status Codes (Os códigos de resposta)

Eles indicam o que aconteceu com a sua requisição. São divididos em 5 classes

1xx: Informativo (Processo em andamento)		
Indica que a requisição foi recebida e o processo continua. É raro ver esses no dia a dia do desenvolvimento web comum.		
100	Continue	O servidor recebeu os cabeçalhos e o cliente pode enviar o corpo.
101	Switching Protocols	O cliente pediu para mudar de protocolo (ex: migrar de HTTP para WebSocket).

2xx: Sucesso (Tudo certo!)		
A ação foi recebida, compreendida e aceita.		
200	Ok	Sucesso padrão.
201	Created	Sucesso e um novo recurso foi criado (muito usado em POST).
204	No Content	Sucesso, mas não há corpo na resposta (comum em DELETE).

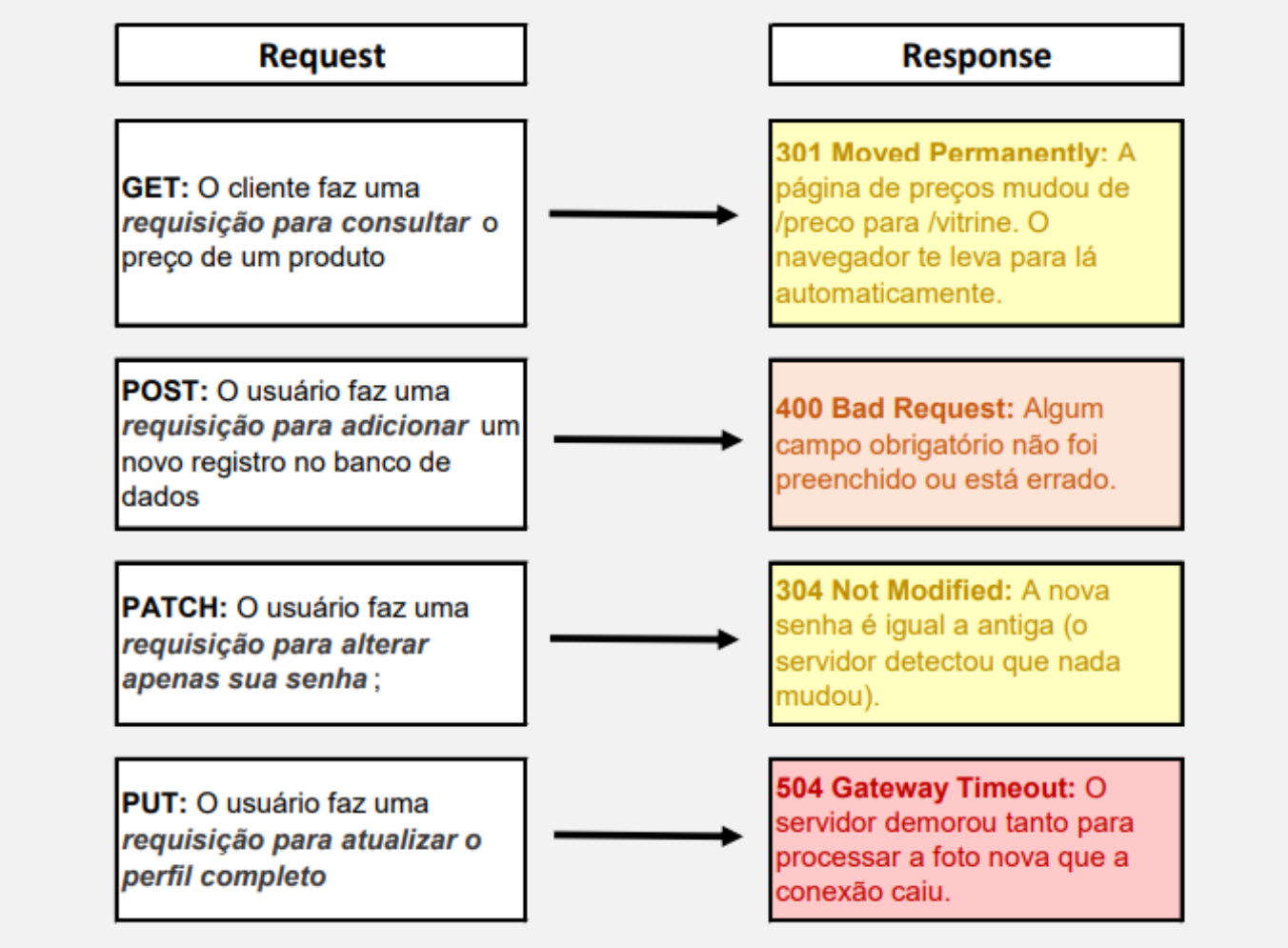
3xx: Redirecionamento (Vá para outro lugar)		
O cliente precisa tomar uma ação adicional para completar a requisição.		
301	Moved Permanently	A URL mudou para sempre.
302	Found (Temporary Redirect)	A URL mudou temporariamente.
304	Not Modified	Usado para cache. Diz que o arquivo não mudou, então o navegador pode usar a versão que já tem salva.

4xx: Erro do Cliente (Você fez algo errado)		
O cliente precisa tomar uma ação adicional para completar a requisição.		
400	Bad Request	A requisição está malformada (JSON errado, por exemplo).
401	Unauthorized	Você não está logado ou não tem permissão.
403	Forbidden	Você está logado, mas não tem permissão para acessar aquele recurso específico.
404	Not Found	O recurso não existe.
405	Method Not Allowed	Você tentou um POST em uma rota que só aceita GET.
429	Too Many Requests	Você está fazendo requisições rápido demais (limite de taxa).

5xx: Erro do Servidor (O servidor falhou)		
A requisição era válida, mas o servidor falhou ao processá-la.		
500	Internal Server Error	Erro genérico (o código do back-end "quebrou").
502	Bad Gateway	O servidor que age como intermediário recebeu uma resposta inválida.
503	Service Unavailable	O servidor está sobrecarregado ou em manutenção.

504	Gateway Timeout	O servidor demorou demais para responder.

Exemplo de requisições e respostas HTTP



⚖️ O que é Idempotência?

Uma operação é considerada **idempotente** quando realizar a mesma requisição várias vezes tem exatamente o **mesmo efeito** que realizá-la apenas uma vez. O estado do servidor não muda após a primeira execução bem-sucedida.

1. GET (Idempotente)

Se você consultar o preço de uma camiseta 10 vezes, o preço continua o mesmo e você não alterou nada no banco de dados.

- **Ação:** Consultar.
- **Resultado:** Sem alteração no servidor.

2. PUT (Idempotente)

Se você enviar uma requisição para atualizar o perfil do usuário para Nome: Ana, Idade: 23, não importa se você enviar isso 1 vez ou 100 vezes. Ao final, o registro no banco será exatamente Nome: Ana, Idade: 23.

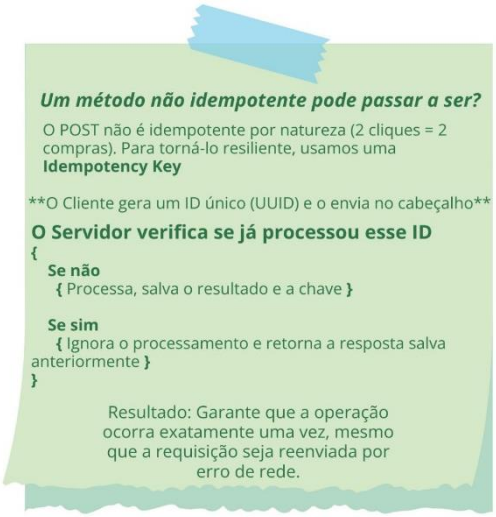
- **Ação:** Substituir.
- **Resultado:** O estado final é sempre o mesmo.

3. DELETE (Idempotente)

Se você deletar o produto ID 50, na primeira vez ele é removido (Status 204 ou 200). Se você tentar deletar de novo, ele continua não existindo. O resultado final (o produto não estar lá) não muda.

- **Ação:** Remover.
- **Resultado:** Uma vez deletado, permanece deletado.

4. POST (NÃO Idempotente)

Aqui é onde mora o perigo. Se você enviar um POST para "Registrar Venda", cada vez que você enviar, o servidor criará uma **nova** linha no banco de dados com um ID diferente.

- **Ação:** Criar/Adicionar.
- **Resultado:** 10 requisições = 10 vendas criadas.